

hw5: page table magic – deduplication and copy-on-write

Due by noon, Friday Oct 27, 2023

In this homework, we implement two advanced virtual memory features in xv6. First, we introduce a deduplication system, wherein the kernel scans the system for duplicated pages and frees up memory by reusing **identical physical memory page frames in several virtual memory locations**.

This, however, introduces a major problem should a process try to write to one of these shared pages. Thus, the second part of our homework is to handle this problem using a copy-on-write approach: write-protect the shared pages, and make a new writable copy for any process that needs it.

Overview

As usual, create your hw5 turn-in repository using this link: <https://classroom.github.com/a/qNj1ej8q>. Clone the hw5 template from the public repository, using the hw5 branch.

The changes mostly live in kalloc.c. Pay attention to the new functions krelease() and kretain(), as well as a new array of struct frameinfo. Here, kretain() increases the reference count on a page (indicated using the virtual address of the kernel's mapping of the physical frame, aka P2V(framenumber<<12), just like how kalloc() and kfree() do it. krelease() decreases the count, and if the count reaches zero, it calls kfree() on the page. kalloc() sets the reference count of a freshly allocated page to 1. Finally, all uses of kfree() throughout xv6 were replaced with krelease().

Attention: whenever you see the term "frame" in xv6, it's about *physical pages*.

What's primarily missing is the implementation of **dedup** and **copy-on-write**, for which placeholders exist in vm.c.

You only need to edit vm.c to achieve a fully-functional implementation. However, it's okay to edit any other files.

Part 1: Deduplication (70%)

The program dedup_reader allocates a lot of memory and fills it with lots of identical content. This uses up a lot of physical RAM. It then calls the new system call sys_dedup(), which identifies duplicate pages, and frees up most of the memory through virtual memory deduplication. The program then reads from the memory to make sure it still works as expected.

A correct solution finishes dedup in less than 1 second without crashing, and shows an increase in the number of free system pages commensurate with the size of the large allocation in the beginning of the program.

The skeleton dedup() function has been provided at the bottom of vm.c. The new system call "dedup" will come here and print out that message. You only need to implement dedup() to finish this part.

Two helper functions (update_checksum() and frames_are_identical()) in kalloc.c are provided for finding the duplicates quickly. Use these functions with your discretion. You can get full points **without** using them, as long as it works as expected.

If your implementation runs correctly but takes a while to execute, dedup() might be repeatedly performing some expensive computations.

Part 2: Copy-on-Write (30%)

The program dedup_writer is similar to dedup_reader, except it then writes to some parts of the memory, checks that other parts of memory are unaffected, calls sys_dedup(), writes some more, and calls sys_dedup() again.

For this to work correctly, you need to **write-protect** your deduped page table entries, so that a page fault is triggered when a write occurs. This is then caught by a new case in the big trap.c switch statement, and which leads to the copyonwrite() skeleton function in vm.c. You need to implement copyonwrite() to allow write on the deduped pages by creating a private copy of that page and map it there with write enabled.

```
cpu1: starting
cpu0: starting
init: starting sh
$ dedup_reader
Freepages at start: 56803
Freepages after malloc: 54357
Freepages after dedup: 56796
$ dedup_writer
Freepages at start: 56803
Freepages after malloc: 54357
Freepages after dedup: 56797
Freepages after writing a little: 56553
Freepages after dedup: 56795
Freepages after writing the rest: 54599
Freepages after dedup: 56796
$ _
```

Note that you may see different numbers but similar patterns--more free pages after each call to dedup.

Bonus challenge: enable copy-on-write for fork()

A correct copy-on-write implementation (Part 2) is required for this part.

fork() is a core functionality in unix-like systems. It creates an identical copy of the calling process. Combined with a consecutive exec(), who replaces the current process with a given program, a new process can be created. Since exec() will be discarding the old context quickly, creating a duplicate process seems to be very expensive. To minimize this cost, real-world fork adopts copy-on-write, which effectively eliminates the actual copying in fork().

Your task is to enable copy-on-write for xv6's fork(). To get started, read the code from proc.c:fork() to **vm.c:copyuvm()**. kalloc() is used in copyuvm() to duplicate process memory. You need to change the implementation in copyuvm() so that it get rid of kalloc(). Similar to that in part-2, the actual copying will happen when the shared page is being written to.

In this part, you only need to employ COW for the user pages. Specifically, the page table is still fully duplicated in a fork. In a real-world implementation, the page table is also duplicated using COW, which further reduces the copying efforts (as small as copying only one page in a fork!). However, that will require more extra work in the page table management code.

```
cpu1: starting
cpu0: starting
init: starting sh
$ cow_fork
Freepages before malloc: 56803
Freepages after malloc: 56680      data: ?
Child: freepages before memset: 56674
Child: freepages after  memset: 56551      data: C
Parent: freepages before memset: 56551
Parent: freepages after  memset: 56551      data: P
Parent: freepages after wait(child): 56680
$
```

of free pages is reduced after the child calls memset.

Requirements

The programs must run without crashing or reporting errors. System free memory should be maximized by the dedup functionality: the reduction in memory usage for a correct program is similar to the size of the full allocation (10 MB) since we fill it with all the same data.

Finally, make sure there are no kernel memory leaks: total system free pages may not change after repeatedly running the test programs.

Turn-in

To verify the correctness of your submission, clone the turn-in repo in a separate directory, build, and test.

X	Prot.	Ignored	Rsvd.	Address of 4KB page frame	Ign.	G	P	A	D	A	P	P	P	R		PTE:
D	Key ⁴						T				C	W	U	/	1	4KB page
											D	S	S	W		

dedup:

- dedup() finds duplicated pages of the current process. Everytime two pages at X and Y are found to contain the identical contents, dedup() releases one page and map the other page to both X and Y. X and Y are two virtual addresses of the current process.
- Example:
 - Before dedup: X==0x4000, PA(X)== **0x10000**; Y==0x6000, PA(Y)==**0x18000**; memcmp(X, Y, PGSIZE) == 0
 - After dedup: X==0x4000, PA(X)== **0x10000**; Y==0x6000, PA(Y)==**0x10000**; physical page at PA==**0x18000** is returned to the freelist.

The process memory starts from 0 to proc->sz. See exec() for the initial size and growproc() for the resizing.

Note that the first page at VA==0 is not mapped. Your code needs to safely ignore those invalid pages.

kretain() increases the reference count of the page by one.

krelease() does the opposite. In addition, if the reference happened to become zero, the page is freed by calling kfree().

krelease() and kretain() automatically maintains the reference count of the each page (using pageframes), and free them when needed.

copy-on-write:

- After dedup, some page are properly mark by the OS as **COW**, but exposed to the **CPU** as "read-only".
- Upon a write to a COW page, the CPU thinks it's an access violation because PTE_W is not set. A page fault exception is then raised and handled by the kernel.
- Note that the bit PTE_COW is ignored by the CPU (Ign. in the above figure). The OS can freely use this bit for bookkeeping tasks.
- the function copyonwrite() in vm.c handles the exception. the trap() function in trap.c dispatches the exception handling.
- copyonwrite() checks if the accessed page is a COW page. If yes, it allocates a new page, duplicates the data in the original page, and grants it write permission by restoring it's PTE_W bit in the page table.

FAQs

How does the CPU distinguish between a copy-on-write page and a read-only page?

- The CPU has no idea what it is. It only looks at the PTE_W for write permission. This is why there will be an exception.

How does the kernel know it?

- There are a few unused bits in the PTE: shown as "ignored" in the CPU [manual](#) (Table 4-19 or Figure 4-11). Those bits can be freely used by software (the OS).
- Define PTE_COW below other PTE_* flags in mmu.h (already provided)

Useful functions/macros:

- PGROUNDUP/PGROUNDNDOWN
- walkpgdir (with the last parameter==0, it returns NULL if page is not mapped)
- krefcount/update_checksum/frames_are_identical
- P2V/V2P/uva2ka There are three different kinds of addresses: **user VA**, **kernel VA**, and **PA**. You need to do conversions between them when necessary.
- PTE_ADDR/PTE_FLAGS
- kalloc/krelease/kretain
- switchuvm/lcr3
- memmove